DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi,
Approved by AICTE and UGC, Accredited by NAAC with 'A' grade & ISO 9001–2015 Certified Institution)
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560 111, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING



Assignment Report

on

Synthetic Lane-Image Dataset for Adversarial Robustness in Autonomous Vehicles

Assignment for Data Science 22AI51

Submitted by

Pratham K	**1DS24AI037**
Spoorthi	**1DS24AI051**

Guided by

Dr. Kusumika Krori Dutta

Department of Artificial Intelligence & Machine Learning
DSCE, Bengaluru



VISVESVARAYA TECHNOLOGICAL UNIVERSITY
JNANASANGAMA, BELAGAVI-590018, KARNATAKA, INDIA
2026-27

Title of the Paper to Be Published

Working title: Perception-Corrupted Trajectory Tracking: Robustness of an Adversarially Trained LQR–Residual-RL Controller under Image-Based Attacks on Lateral-Error Estimates

Problem Statement

Autonomous-vehicle trajectory controllers are usually evaluated with clean state measurements. In practice the lateral error e_y and heading error ε_ψ come from perception, which can be corrupted by adversarial perturbations (FGSM, PGD) and by natural conditions (brightness, glare, shadows, blur, noise). To the best of our review, the robustness of an adversarially trained controller such as that of Lee et al. [1] under such corrupted state estimates has not been evaluated [2], [3]. The problem addressed in this assignment is the data part of that study: to build a labelled lane-image dataset with exact ground truth, so that a CNN can estimate e_y and ε_ψ from images and the effect of attacks on these estimates, and through them on closed-loop tracking, can be measured.

Data Life Cycle

The full life cycle that the project follows is shown below. Stages 1 to 5 are complete and are reported in this document; stages 6 to 9 follow after the dataset submission.

Table 1. Data life cycle

#	Stage	What is done	Status
1	Problem definition	Research gap and objective stated (above)	Done
2	Data generation	MATLAB simulator; two members, 15,000 images each, fixed seeds	Done
3	Data description and exploration	Types, schema, counts, label distributions, sample images	Done
4	Validation and cleaning	Completeness, duplicate, range, leakage and consistency checks	Done
5	Preprocessing, splitting and storage	Run-wise train / val / test split; merged dataset with manifests	Done
6	Model building	CNN estimating e_y and ε_ψ from images (pixel scaling, label standardisation)	To follow
7	Attack evaluation	FGSM [4], PGD [5], brightness, glare, shadows, blur, noise	To follow
8	Closed-loop evaluation	Attacked estimates fed to the LQR-based controller of [1]	To follow
9	Reporting and publication	Results, comparison with baselines, paper	To follow

1. DATA DESCRIPTION

1.1 Type of data

The dataset has two linked parts: 30,000 grayscale lane images (8-bit PNG, 64 rows × 128 columns) and one structured table (labels.csv) with 30,000 rows × 19 columns, one row per image. The images are the model input; e_y and ε_ψ are continuous regression targets and speed is an auxiliary label. Rows are grouped into runs of 50 consecutive frames taken 0.1 s apart, so the data is grouped time-series data rather than independent samples. All of it is synthetic and numerical; no text or real-world sensor data is involved.

Table 2. Attribute types in labels.csv

Attribute type	Attributes	Count
Numeric, continuous	ey_m, epsi_rad, speed_mps, s_m, curvature_1pm, lane_width_m, line_brightness, road_brightness, bg_brightness, sky_brightness, texture_amp	11
Numeric, discrete (integer)	member_id, run_id, frame, texture_scale	4
Categorical (nominal)	road (S, T, O, U, Random), split (train, val, test_seen, test_unseen)	2
Binary	dashed_left	1
Identifier (text)	filename	1
Total		**19**

1.2 Number of datasets and their sources

There are two source datasets, one per team member, which were merged into one final dataset. Both were generated by the same MATLAB generator (R2026a) with the same shared configuration; only the random seed, the file prefix and the run_id range differ. **No external or real-world dataset, website or download was used.** The sources of the data are therefore the generator itself and the vehicle model and road definitions inside it, whose approach follows Lee et al. [1].

Table 3. Datasets

Dataset	Source	Images	run_id range
Member 1 (LaneDataset_Member1)	MATLAB generator, seed 1042	15,000	1 to 300
Member 2 (LaneDataset_Member2)	MATLAB generator, seed 2042	15,000	10001 to 10300
Merged (LaneDataset_Merged)	Merge of the two above after checks	**30,000**	both

How the data is generated, in brief: a linearised bicycle model (9.72 m/s) is driven by an LQR (Q = I, R = 1, 0.01 s) along five road shapes (S, T, O, U and a seeded random shape). Each run lasts 5 s and starts from random offsets (e_y within ±1.5 m, ε_ψ within ±0.2 rad) with random steering pushes and kicks. Every 0.1 s a pinhole camera (1.4 m high, pitched 5°, 90° field of view) renders the lane from the true pose, with randomised lane width (3.2 to 3.8 m), line, road, background and sky brightness, texture, and a dashed or solid left line. Labels come directly from the simulated pose, so they are exact.

1.3 Missing data

Missing data was measured on the merged labels.csv: 30,000 rows × 19 attributes = 570,000 values, of which **0 are missing (0%)**. All 30,000 image files listed in the table exist, so no images are missing either. The data is complete because every value is computed for every frame by the simulator, and the merge script stops with an error if any value is missing or non-finite. No missing values were introduced artificially.

Table 4. Attributes by level of missing data

Missing-data level	Attributes	Records affected
0% (complete)	19	0
up to 10%	0	0
20%	0	0
30%	0	0
40%	0	0
50%	0	0
60%	0	0
more than 70%	0	0

1.4 Common attributes

All 19 attributes are common to the Member 1 dataset, the Member 2 dataset and the merged dataset: the column names, order, units and types are identical, which the merge script verifies before merging. The units are metres for distances, radians for angles and metres per second for speed, and brightness values are on a 0 to 1 scale. The attribute groups are listed in Table 2. The same structure is also common across the four splits and the five roads.

Table 5. Split composition of the merged dataset

Split	Roads	Runs	Images	Images per road
train	S, T, O, U	360	18,000	4,500
val	S, T, O, U	80	4,000	1,000
test_seen	S, T, O, U (separate runs)	80	4,000	1,000
test_unseen	Random only	80	4,000	4,000
Total		**600**	**30,000**	

2. DATA CLEANING METHOD SELECTION AND JUSTIFICATION

Because the data is generated, it contains no sensor noise, typing errors or missing records, so the cleaning here is mainly verification: each possible data problem is checked, and a treatment is chosen only where the check shows a problem. The methods below were selected because the usual treatments (imputation, deleting outliers, smoothing) would damage the exact ground truth that makes this dataset useful.

Table 6. Cleaning methods selected and why

Issue	Finding	Method selected	Justification
Missing values	0 of 570,000 values; no missing images	Completeness check; no imputation or deletion	Imputing a label would replace exact ground truth with an estimate, and there is nothing to delete
Duplicates	No duplicate filenames; no run shared between members	Uniqueness check on filename and (member_id, run_id); separate run_id ranges	Duplicates across splits would inflate test results
Outliers	e_y within ±1.5 m, ε_ψ within about ±0.3 rad, speed 8.7 to 10.7 m/s; all physically valid	Range check; no values removed	Extreme values are real states created on purpose (clamped kicks); removing them would reduce coverage of large errors
Data leakage	Each run belongs to exactly one split	Split by run, before any frame is saved	Neighbouring frames are almost identical, so a frame-wise split would leak
Skewed labels	Pilot: 59 to 69% of frames had $\lvert e_y \rvert < 0.1$ m	Regenerate with random kicks, instead of resampling or reweighting	Resampling would repeat near-identical frames; new data adds real diversity (final: 14 to 16%)
Inconsistent format or units	None	One schema, metres and radians, labels rounded to 6 decimals, lossless PNG	Identical results on both machines (matching checksums)
Road imbalance	Exactly 4,500 / 1,000 / 1,000 images per road in train / val / test_seen	Balanced by design (same number of runs per road)	Avoids bias towards one road shape
Scaling	Not yet applied	Planned: pixels ÷ 255; labels standardised with training-split statistics only	Helps CNN training; using only training statistics avoids leakage

3. RESULTS

3.1 Verification results

Before the full run, both members generated 20 images with a fixed test seed (777); the label checksum (1373032419) and image checksum (172617928) were identical on both machines (MATLAB R2026a Update 4). The merge script then passed all checks on the final data: counts of 18,000 / 4,000 / 4,000 / 4,000 per split and 15,000 per member, no duplicates, no run in more than one split, the random road only in test_unseen, every listed image present, no missing or non-finite values, and identical configuration values from both members (one non-blocking warning: the text of the saved configuration file differed, the values did not).

3.2 Label statistics

Table 7. Label statistics of the merged dataset (near zero means $\lvert e_y \rvert < 0.1$ m or $\lvert \varepsilon_\psi \rvert < 0.01$ rad)

Split	Images	e_y range (m)	e_y std (m)	ε_ψ range (rad)	ε_ψ std (rad)	Speed (m/s)	Near zero e_y / ε_ψ
train	18,000	−1.500 to 1.495	0.506	−0.313 to 0.270	0.111	8.72 to 10.72	14% / 6%
val	4,000	−1.500 to 1.440	0.496	−0.274 to 0.267	0.110	8.73 to 10.72	16% / 6%
test_seen	4,000	−1.500 to 1.500	0.510	−0.289 to 0.290	0.115	8.74 to 10.71	14% / 6%
test_unseen	4,000	−1.500 to 1.489	0.506	−0.286 to 0.283	0.111	8.72 to 10.72	15% / 6%

Figure 1 shows broad, bell-shaped distributions of e_y and ε_ψ in all four splits, with very similar spread, so the test sets are comparable with the training data in label range. Figure 2 shows sample images from the S, T and U roads.

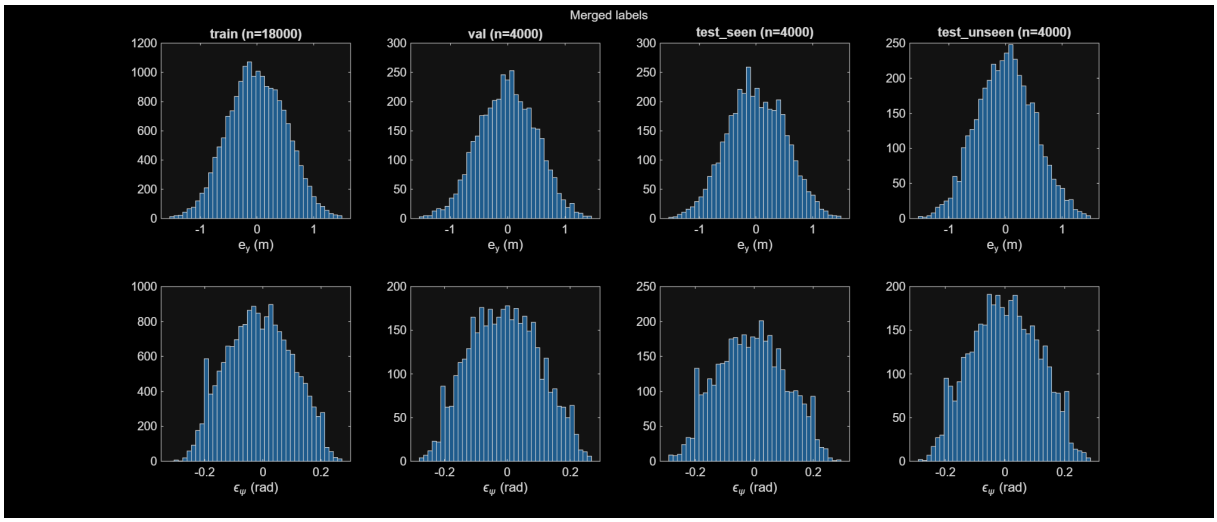


Figure 1. Histograms of lateral error e_y (top row) and heading error ε_ψ (bottom row) for the four splits of the merged dataset.

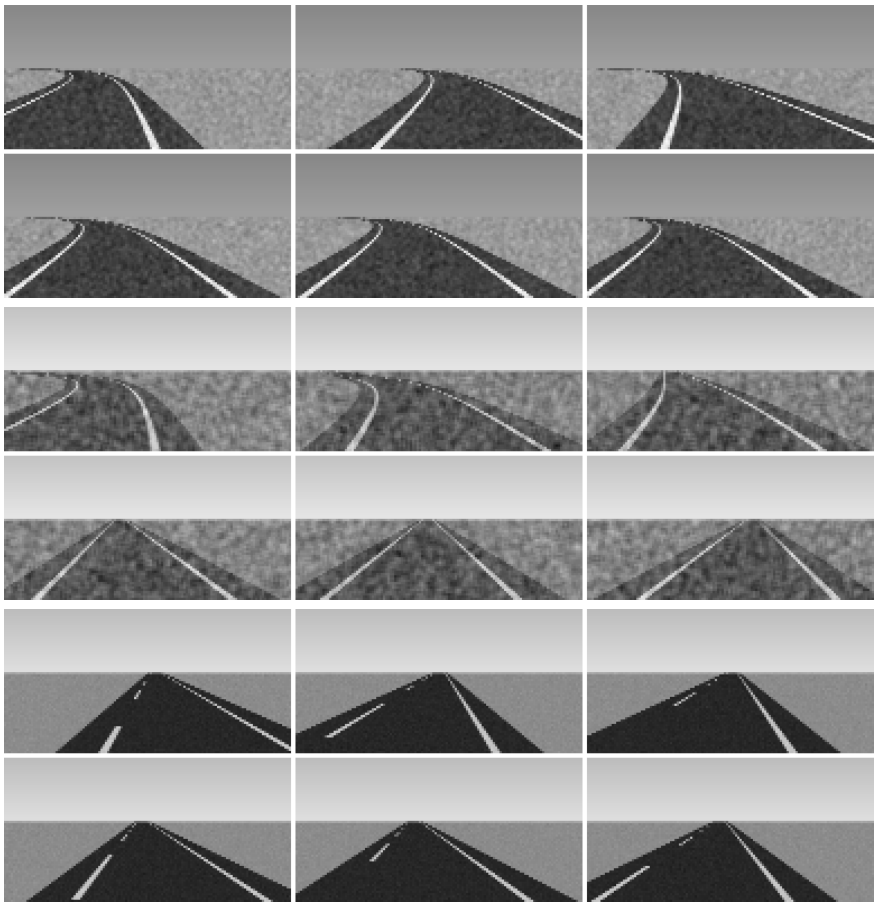


Figure 2. Six frames spread across one 5 s training run on the S road (top), T road (middle) and U road (bottom), enlarged 2×.

3.3 Effect of the cleaning step on the label distribution

The table compares the 1,300-image pilot (before the random kicks) with the final dataset. The kicks reduced the share of near-zero e_y frames from about two thirds to about one sixth and doubled the spread of e_y and ε_ψ.

Table 8. Pilot (before kicks) versus final dataset

Split	e_y near zero, pilot	e_y near zero, final	e_y std, pilot (m)	e_y std, final (m)	ε_ψ std, pilot (rad)	ε_ψ std, final (rad)
train	64%	14%	0.280	0.506	0.0620	0.111
val	69%	16%	0.214	0.496	0.0546	0.110
test_seen	66%	14%	0.255	0.510	0.0624	0.115
test_unseen	59%	15%	0.273	0.506	0.0562	0.111

4. CONCLUSION

The dataset consists of 30,000 synthetic grayscale lane images with exact e_y, ε_ψ and speed labels, built from two member datasets of 15,000 images each. It is complete (0% missing), has no duplicates, is split by run into 18,000 training, 4,000 validation, 4,000 seen-road test and 4,000 unseen-road test images, and keeps a random-shaped road for the unseen test only. The cleaning step was mainly verification, because the usual treatments would have damaged the exact labels; the one real data problem found, labels clustered near zero, was fixed by regenerating the data with random kicks, which raised the spread of e_y from about 0.25 m to about 0.5 m.

The main limitation is that the data is synthetic, with a simple renderer, so results on it may not transfer to real cameras. The next stages of the life cycle are to train the CNN, apply the attacks and evaluate the attacked estimates in the closed loop.

5. REFERENCES

Sources of the datasets: none external. No dataset was downloaded from any website; all data was generated by the team's MATLAB generator (config.m, generate_dataset.m, merge_datasets.m), which is submitted together with the dataset.

Works cited:

[1] Lee, D., Oh, T., Kim, J., & Yoo, J. (2026). Adversarial-training-based robust reinforcement learning control for autonomous vehicle trajectory tracking. IEEE Access, 14, 43954–43975. https://doi.org/10.1109/ACCESS.2026.3675864

[2] Al Islam, M., & El-Wakeel, A. S. (2026). Adversarial robustness analysis of cloud-assisted autonomous driving systems. arXiv preprint arXiv:2604.04349. https://doi.org/10.48550/arXiv.2604.04349

[3] Mohan, A., Xie, X., Sambandham, V. T., & Schön, T. (2026). Real-time evaluation of autonomous systems under adversarial attacks. arXiv preprint arXiv:2605.03491. https://arxiv.org/abs/2605.03491

[4] Goodfellow, I. J., Shlens, J., & Szegedy, C. (2015). Explaining and harnessing adversarial examples. In Proceedings of the International Conference on Learning Representations (ICLR 2015). arXiv:1412.6572. https://arxiv.org/abs/1412.6572

[5] Madry, A., Makelov, A., Schmidt, L., Tsipras, D., & Vladu, A. (2018). Towards deep learning models resistant to adversarial attacks. In Proceedings of the International Conference on Learning Representations (ICLR 2018). arXiv:1706.06083. https://arxiv.org/abs/1706.06083